Họ và Tên: Vũ Trường Giang
MSSV : 202616017
Ngành : Kỹ thuật Điều Khiển – Tự Động Hóa
Mã lớp : EE2 – 11 – K71

Báo cáo dự án
Hồi quy logistic bằng NumPy

Bài toán phân loại RED và NOT RED

1 Mục tiêu dự án

Dự án triển khai trực tiếp thuật toán hồi quy logistic bằng Python và NumPy nhằm tìm hiểu quá trình một mô hình học các tham số từ dữ liệu có nhãn. Bài toán đặt ra là phân loại một mẫu có ba đặc trưng đầu vào vào một trong hai nhóm: RED được mã hóa bằng nhãn 1 và NOT RED được mã hóa bằng nhãn 0.

Chương trình thực hiện các bước tính đầu ra, đánh giá sai lệch, tính gradient và cập nhật tham số. Phạm vi hiện tại tập trung vào thuật toán huấn luyện trên bộ dữ liệu nhỏ khai báo trong chương trình. Kết quả kiểm tra trên tám mẫu huấn luyện được trình bày riêng với các yêu cầu đánh giá trên dữ liệu mới.

2 Công cụ và dữ liệu

Chương trình sử dụng Python và NumPy để biểu diễn dữ liệu bằng mảng, thực hiện phép nhân ma trận và tính toán trên toàn bộ tập dữ liệu. Thuật toán huấn luyện được triển khai trực tiếp, không gọi mô hình có sẵn từ thư viện học máy.

Dữ liệu gồm 8 mẫu, mỗi mẫu có 3 đặc trưng, trong đó có 3 mẫu RED và 5 mẫu NOT RED. Ba đặc trưng được ký hiệu là x_1, x_2 và x_3. Bộ giá trị đầu vào và nhãn của từng mẫu được trình bày trong bảng kết quả ở mục 7.

Đại lượng	Kích thước	Ý nghĩa
X	(8, 3)	Ma trận đặc trưng của 8 mẫu.
y	(8,)	Nhãn thực tế của từng mẫu.
w	(3,)	Ba trọng số mô hình cần học.
b	Một số thực	Hệ số dịch chuyển của mô hình.
p	(8,)	Đầu ra dự đoán cho 8 mẫu.

Dữ liệu đầu vào có các giá trị nằm trong khoảng 0 đến 1. Tuy nhiên, mã hiện tại không có bước chuẩn hóa dữ liệu; việc lựa chọn hoặc biến đổi các giá trị này diễn ra trước khi đưa vào chương trình.

Để tái lập bài toán trên dữ liệu đo, hồ sơ dữ liệu cần nêu rõ ý nghĩa ba đặc trưng, nguồn thu thập và cách tiền xử lý. Trong phạm vi báo cáo, các cột được giữ tên x_1, x_2 và x_3 để phản ánh đúng cấu trúc số học của mã nguồn.

3 Cấu trúc mô hình

Với một mẫu đầu vào, mô hình trước tiên tính tổ hợp tuyến tính của ba đặc trưng:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Trọng số w_j quyết định mức tác động của đặc trưng x_j, còn bias b dịch chuyển giá trị đầu ra tuyến tính. Giá trị z sau đó được đưa qua hàm sigmoid:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid đưa đầu ra về khoảng từ 0 đến 1. Trong bài toán này, p được mô hình diễn giải là xác suất thuộc lớp RED. Khi $z = 0$, đầu ra bằng 0,5; khi z tăng, đầu ra tiến gần 1 .

Trong chương trình, $x @ w + b$ tính đồng thời giá trị z cho toàn bộ các mẫu. Đây là vector hóa, tức biểu diễn phép tính bằng thao tác trên mảng thay vì viết vòng lặp riêng cho từng mẫu.

4 Hàm mất mát và thành phần L2

Hàm mất mát, hay loss, là đại lượng đo mức độ không phù hợp giữa dự đoán và nhãn thực tế. Chương trình sử dụng binary cross-entropy, viết tắt là BCE:

$$L_{BCE} = -\frac{1}{m}\sum_{i=1}^{m}[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

Với mẫu RED, dự đoán p càng gần 1 thì mất mát càng nhỏ. Với mẫu NOT RED, dự đoán càng gần 0 thì mất mát càng nhỏ. Một dự đoán sai với mức tự tin cao sẽ bị phạt mạnh .

Chương trình dùng numpy.clip() để giới hạn xác suất trong khoảng từ 10^{-12} đến $1 - 10^{-12}$ trước khi lấy logarithm. Cách này tránh phép tính $ln(0)$. Hàm mục tiêu đầy đủ trong mã là:

$$J = L_{BCE} + \frac{\lambda}{2}\sum_{j=1}^{3} {w_j}^2$$

Thành phần thứ hai là điều chuẩn L2: thêm một khoản phạt khi trọng số có độ lớn cao. Trong mã, $\lambda = 0{,}015$ và khoản phạt chỉ áp dụng cho w, không áp dụng cho b. L2 giúp kiểm soát độ lớn trọng số, nhưng khả năng dự đoán tốt trên dữ liệu mới vẫn cần được đánh giá riêng.

5 Thuật toán huấn luyện

Chương trình sử dụng gradient descent, tức cập nhật tham số theo hướng ngược với gradient để tìm cách giảm hàm mục tiêu. Gradient biểu diễn mức độ thay đổi của hàm mục tiêu theo từng tham số. Mỗi lần cập nhật sử dụng toàn bộ tám mẫu, tương ứng phương pháp full-batch gradient descent.

Các công thức được triển khai là:

$$\frac{\partial J}{\partial w} = \frac{X^T(p - y)}{m} + \lambda w, \quad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(p_i - y_i)$$

Sau đó, tham số được cập nhật theo:

$$w_{k+1} = w_k - \alpha\frac{\partial J}{\partial w}, \quad b_{k+1} = b_k - \alpha\frac{\partial J}{\partial b}$$

Trong đó α là learning rate, hay tốc độ học, quyết định độ lớn của mỗi bước cập nhật. Các tham số huấn luyện được đặt như sau:

Tham số	Giá trị
Trọng số ban đầu	[0, 0, 0]
Bias ban đầu	0,0
Learning rate	0,15
Hệ số L2	0,015
Số vòng lặp tối đa	1000
Ngưỡng biến thiên loss để dừng	0,0001

6 Điều kiện dừng

Chương trình dừng khi độ chênh lệch giữa hai giá trị hàm mục tiêu liên tiếp thỏa mãn:

$$|J_k - J_{k-1}| \leq 10^{-4}$$

Điều kiện này cho biết hàm mục tiêu thay đổi ít giữa hai lần đánh giá. Nó không có nghĩa là loss đã nhỏ hơn 0,0001, cũng không chứng minh đã đạt nghiệm tối ưu chính xác.

Biến best_loss đang đóng vai trò ngưỡng thay đổi cho phép. Đây là điều kiện dừng dựa trên hàm mục tiêu huấn luyện; chương trình chưa theo dõi tập dữ liệu xác thực riêng để quyết định dừng.

7 Kết quả kiểm tra bằng Python và NumPy

Chạy lại mã nguồn với đúng dữ liệu và tham số đã cung cấp trong môi trường Python 3.13.13, NumPy 2.3.5 cho kết quả xấp xỉ:

$$w = \begin{bmatrix} 2{,}508978 \\ -1{,}406611 \\ -1{,}832773 \end{bmatrix}, \quad b = -0{,}515372$$

Thuật toán dừng tại epoch = 292, theo cách đánh số từ 0. Tại thời điểm đó đã thực hiện 292 lần cập nhật tham số và 293 lần đánh giá loss. Số lần cập nhật được đối chiếu bằng vòng lặp theo dõi riêng, giữ nguyên công thức và thứ tự xử lý của hàm train().

Đại lượng	Giá trị xấp xỉ
Hàm mục tiêu ban đầu	0,693147
BCE tại thời điểm dừng	0,220776
Thành phần phạt L2	0,087244
Tổng hàm mục tiêu tại thời điểm dừng	0,308020
Chênh lệch loss ở lần kiểm tra cuối	0,000099283

Mã gốc kết thúc bằng việc in trọng số và bias. Để đánh giá thêm khả năng phân loại, áp dụng quy tắc RED khi $p \geq 0{,}5$; còn lại là NOT RED. Quy tắc này được bổ sung cho phần đánh giá dưới đây.

Mẫu	Đầu vào $x_1\ x_2\ x_3$	Nhãn thật	p RED	Nhãn dự đoán
1	(0,90; 0,10; 0,10)	1	0,8051	1
2	(0,80; 0,20; 0,10)	1	0,7364	1
3	(0,75; 0,15; 0,20)	1	0,6876	1
4	(0,10; 0,85; 0,15)	0	0,1499	0
5	(0,20; 0,70; 0,20)	0	0,2035	0
6	(0,15; 0,20; 0,90)	0	0,1121	0
7	(0,20; 0,15; 0,80)	0	0,1557	0
8	(0,30; 0,30; 0,75)	0	0,1738	0

Mô hình phân loại đúng 8/8 mẫu huấn luyện với ngưỡng này. Kết quả thể hiện khả năng phân tách bộ dữ liệu đã học; chưa có phép thử độc lập để xác định hiệu quả trên mẫu mới.

Trọng số thứ nhất dương, hai trọng số còn lại âm. Với các đặc trưng khác được giữ cố định, tăng x_1 làm tăng đầu ra RED, còn tăng x_2 hoặc x_3 làm giảm đầu ra đó.